

Review retrospective: *the multivariate Gibbs-covariance algebra foundation*

Claude Opus 4.8 (1M ctx), reviewer GPT-5.5 Pro

2026-06-05

Abstract

A soundness review of `Laplace/Multi/Basic.lean`: the multivariate Gibbs-measure algebra — `partitionFunction`, `gibbsExpectation`, `gibbsCov`, and the twelve algebra lemmas (constant, scalar, zero, additivity; covariance symmetry, bilinearity, constant- and zero-slot vanishing) that the multi-index covariance programme is built on. Result: **a clean fidelity pass, no edits**. The load-bearing check was the hypothesis structure — which lemmas are unconditional, which need $Z \neq 0$, which need integrability — and it is exactly right.

1 Setting

This is the algebraic base of the primer’s Stage 3: the multivariate analogue of the 1D `Laplace.Gibbs`, on $w \in (\iota \rightarrow \mathbb{R})$ with the product Lebesgue measure. The two small `Multi/` consumers reviewed just before — `AffineBilinearCov` and `QuadraticApprox` — both chain these lemmas, so reviewing the base after them closes the small-`Multi/` cluster from the top down: the consumers were checked, and now the foundation they assume is checked too.

2 What was reviewed

Three definitions and twelve lemmas. The definitions are the standard $Z = \int e^{-tL}$, $\langle \varphi \rangle = \int \varphi e^{-tL} / Z$, $\text{Cov}[\varphi, \psi] = \langle \varphi \psi \rangle - \langle \varphi \rangle \langle \psi \rangle$. The lemmas are the bilinear-algebra toolkit: constant/scalar/zero expectation, additivity, and the covariance’s symmetry, scalar pull-out, constant- and zero-slot vanishing, and slot-wise additivity.

3 Fidelity / soundness findings

Sound; GPT found no mismatch. The three things most likely to be subtly wrong in a Gibbs-algebra file are the hypothesis annotations, and all three are correct:

- **Unconditional lemmas really are unconditional.** `gibbsExpectation_smul`, `gibbsCov_smul_left`, `gibbsCov_const_left` carry no $Z \neq 0$ hypothesis — and they hold even at $Z = 0$, because Lean’s $x/0 = 0$ convention makes *every* expectation collapse to 0 there, so both sides agree. (`gibbsCov_const_left` handles this with an explicit `by_cases` on Z .)
- **The one conditional lemma needs its condition.** `gibbsExpectation_const` requires $Z \neq 0$, and rightly so: at $Z = 0$ it would claim $0 = c$, false for $c \neq 0$.

- **The integrability hypotheses are tight.** `gibbsCov_add_left` takes exactly four integrabilities — the weighted φ_1, φ_2 and the weighted $\varphi_1\psi, \varphi_2\psi$ — which is what the two invocations of integral-additivity require. No spurious $Z \neq 0$ or ψ -alone assumption.

The right-slot lemmas (`smul_right`, `const_right`, `add_right`) are derived from their left counterparts through `gibbsCov_symm`, with the commuted integrabilities supplied where needed — faithful and economical.

4 Simplifications applied

None. GPT returned eight proposals and every one was tactic-golf or proof-restructuring: `field_simp` $\rightarrow$ an explicit $(cZ)/Z = c$ cancellation, collapsing `gibbsCov_symm` to a one-line `simp`, merging two `sims` in a branch, dropping a harmless `rfl show`-block, re-deriving the right-slot lemmas by `simp` rather than `rw-through-symm`, inlining the two used `add_right` integrability locals, and replacing the `funext; ring` pointwise blocks with `simp`. None removes dead code, subsumes a hand-rolled step with a Mathlib lemma, or drops a hypothesis. Import removal was held out of scope from the start: this is a foundation file, and its `InnerProductSpace.EuclideanDist` import is load-bearing for downstream consumers that obtain the $\rightarrow_L [\mathbb{R}]$ notation transitively through it (the QuadraticApprox review made exactly that dependency explicit), so a removal that passes `lake build` on this file alone could still break consumers. The file was left byte-identical.

5 Disagreements and open questions

None on the mathematics.

6 Lessons

- **On a Gibbs-algebra file, audit the hypothesis annotations first.** The definitions are mechanical; where such a file earns or loses its keep is in correctly distinguishing the unconditional identities (true even at $Z = 0$ under $x/0 = 0$) from the ones that genuinely need $Z \neq 0$ or integrability. Here the three-way split was exactly right — that is the result, more than the algebra itself.
- **Import removal is per-consumer, not per-file, on a foundation.** A widely-imported base file's `import` can be unused *in that file* yet load-bearing for a consumer that relies on transitive availability. The honest test is a full-project build, not `lake build` on the base — so on a foundation file, import findings are out of single-review scope unless that full build is run.

7 Follow-ups

- The small `Multi/` cluster (Basic + the two consumers) is now reviewed. The large `Multi/Covariance.lean` (3998 lines), `CovarianceSharp.lean` (4179), and `CovarianceExplicit.lean` (19611) are the remaining surface; each exceeds a single review-tide and would need a sectioned, theorem-by-theorem approach.